

Module « Technologies Web »

LE LANGUAGE PHP

Connexion aux bases de données en PHP avec PDO

Première année du cycle d'ingénieur :
Génie Informatique & Génie Industriel

Pr. Mohamed DAOUDI
Année universitaire 2024-2025

PLAN

I. Introduction à PDO

II. Préparation à la connexion

III. Connexion avec PDO

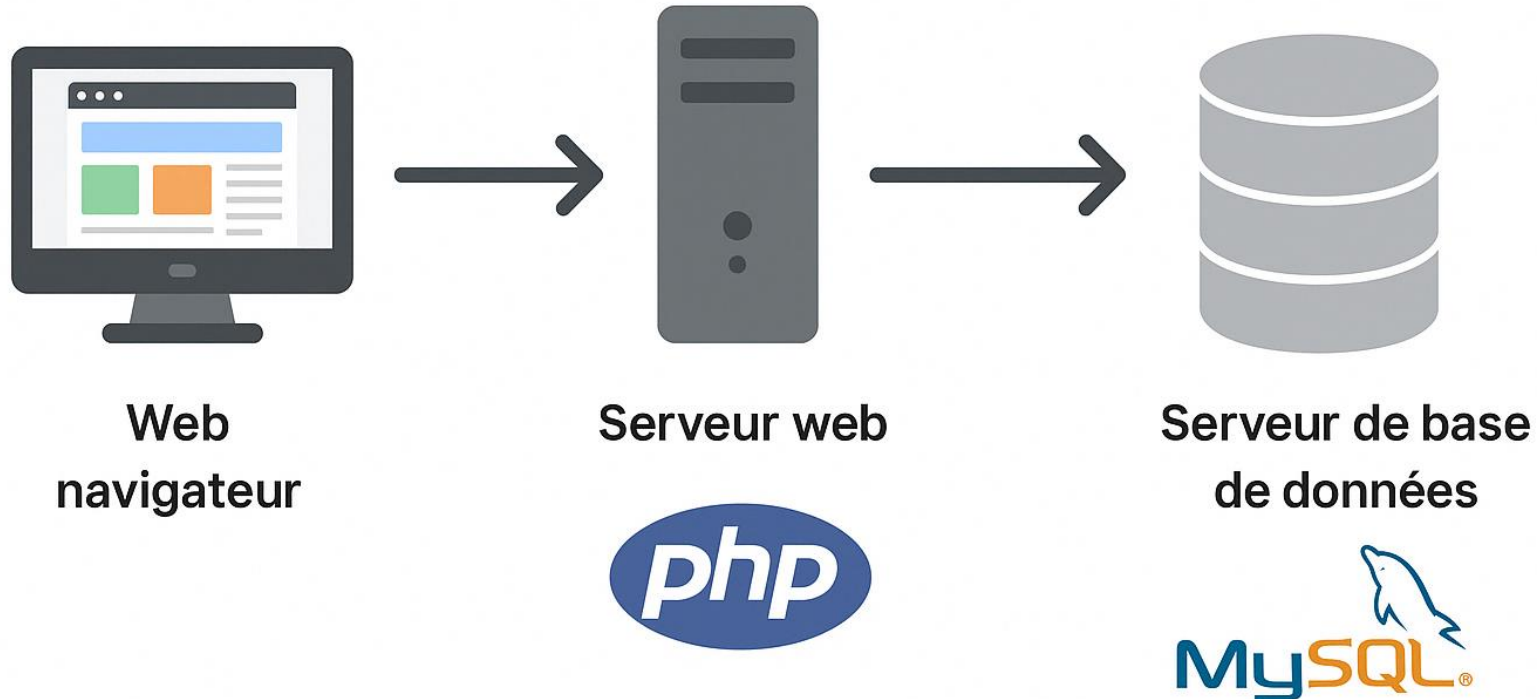
IV. Exécution des requêtes

V. Requêtes préparées

VI. Récupération des résultats

Introduction

Architecture d'un site web avec MySQL/PHP



I. Introduction à PDO

Qu'est-ce que PDO ?

PDO (PHP Data Objects) est une extension (Ecrit en C) fournie par PHP qui permet d'accéder à plusieurs types de bases de données (MySQL, PostgreSQL, Oracle, SQLite, etc.) de manière unifiée et orientée objet.

Elle fournit une interface unifiée pour se connecter, exécuter des requêtes SQL, récupérer des résultats, et sécuriser les échanges avec la base.

La même structure de code peut être utilisée pour MySQL ou PostgreSQL en ne changeant que le DSN (Data Source Name).

I. Introduction à PDO

Activer l'extension PDO dans XAMPP Server

- Allez dans le dossier xampp/php et ouvrez le fichier php.ini avec un éditeur de texte.
- Recherchez les lignes suivantes (pour MySQL) :

```
;extension=pdo_mysql  
;extension=pdo_sqlite
```

- Supprimez le point-virgule ; devant les extensions que vous souhaitez activer :

```
extension=pdo_mysql  
extension=pdo_sqlite
```

- Redémarrer Apache Après avoir enregistré les modifications dans php.ini, retournez dans le panneau XAMPP et redémarrez Apache pour que les changements prennent effet.

Les pilotes PDO pour différents systèmes de gestion de bases de données (SGBD)

PDO (PHP Data Objects) nécessite des pilotes spécifiques pour chaque SGBD. Le tableau présente les pilotes PDO disponibles pour divers moteurs de base de données.

Par exemple :

- Pour se connecter à une base Oracle, on utilise le pilote [PDO_OCI].
- Pour MySQL, on utilise [PDO_MYSQL].
- Pour PostgreSQL, le pilote est [PDO_PGSQL], etc.

Oracle OCI	[PDO_OCI]
ODBC V3, IBM DB2	[PDO_ODBC]
MySQL 3.x	[PDO_MYSQL]
Postgres	[PDO_PGSQL]
SQLite 3.x	[PDO_SQLITE]
Firebird	[PDO_FIREBIRD]

Avantages par rapport à mysqli ou mysql_*

Ce tableau comparatif met en évidence les principales différences entre les trois méthodes de connexion à une base de données en PHP : mysql_* (désormais obsolète), mysqli, et PDO.

Fonctionnalité	mysql_* (obsolète)	mysqli	PDO
Support de plusieurs bases de données	✗	✗ (MySQL uniquement)	✓
Programmation orientée objet	✗	✓	✓
Requêtes préparées (anti-injection SQL)	✗	✓	✓
Portabilité du code	✗	✗	✓
Connexions persistantes	✗	✓	✓
Gestion des erreurs avec exceptions	✗	✓ (partielle)	✓

II. Préparation à la connexion

Avant d'établir une connexion à une base de données avec PDO en PHP, certaines étapes de préparation sont nécessaires. Cela garantit que la base de données est prête à être utilisée et que le code PHP peut y accéder sans erreur.

1. Informations nécessaires à la connexion

Pour que PHP se connecte correctement à la base de données, vous devez connaître les paramètres de connexion suivants :

Information	Description
Hôte (host)	Adresse du serveur MySQL (exemple : localhost en local)
Nom de la base	Nom de la base de données (exemple : ensa)
Utilisateur (user)	Nom d'utilisateur MySQL (exemple : root ou user_ensa)
Mot de passe (passwd)	Mot de passe de l'utilisateur (vide par défaut pour root)

II. Préparation à la connexion

2. Présentation des constantes de connexion

Pour rendre le code plus lisible et facilement modifiable, on déclare généralement ces informations sous forme de constantes en PHP :

- `define('DB_HOST', 'localhost');`
- `define('DB_NAME', 'ensa');`
- `define('DB_USER', 'root');`
- `define('DB_PASS', '');`



Ces constantes seront ensuite utilisées pour construire la chaîne de connexion DSN et instancier l'objet PDO :

```
$dsn = 'mysql:host=' . DB_HOST . ';dbname=' . DB_NAME;
$options = [
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
];
try {
    $db = new PDO($dsn, DB_USER, DB_PASS, $options);
    echo "Connexion réussie à la base de données ensa.";
} catch (PDOException $e) {
    die("Échec de la connexion : " . $e->getMessage());
}
```

III. Connexion avec PDO

1. Présentation des constantes de connexion

Pour se connecter à une base de données MySQL avec PDO, il faut utiliser le constructeur de la classe PDO. Voici la syntaxe de base :

```
$dsn = 'mysql:host=localhost;dbname=ensa';
```

```
$user = 'root';
```

```
$pass = '';
```

```
$db = new PDO($dsn, $user, $pass);
```

`$dsn` (Data Source Name) : indique le type de base (mysql), le serveur (localhost), et la base de données (ensa).

`$user` : nom d'utilisateur (ici root).

`$pass` : mot de passe (vide ici).

III. Connexion avec PDO

2. Connexion avec gestion d'erreurs (bloc try/catch)

Il est fortement recommandé d'encadrer la connexion avec un bloc try/catch pour gérer proprement les éventuelles erreurs de connexion :

```
try {  
    $db = new PDO($dsn, $user, $pass);  
    // Configuration du mode d'erreur en exception  
    $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);  
    echo "Connexion réussie !";  
} catch (PDOException $e) {  
    // Affiche un message d'erreur en cas d'échec de connexion  
    echo "Erreur de connexion : ". $e->getMessage();  
}
```

Si la connexion échoue (mauvais mot de passe, base inexistante...), le message d'erreur détaillé est affiché sans arrêter brutalement le script.

III. Connexion avec PDO

3. Connexion persistante : principe et utilité

Par défaut, chaque connexion PDO est ouverte et fermée à chaque script PHP exécuté. Une connexion persistante reste ouverte et peut être réutilisée par d'autres scripts, ce qui réduit le temps de connexion, surtout sur des sites à fort trafic. Pour l'activer, il suffit de passer une option lors de la création de l'objet PDO :

```
$options = array(
    PDO::ATTR_PERSISTENT => true, // active la connexion persistante
    PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION // gestion des erreurs
);
try {
    $db = new PDO($dsn, $user, $pass, $options);
    echo "Connexion persistante réussie !";
} catch (PDOException $e) {
    echo "Erreur de connexion persistante : ". $e->getMessage();
}
```

IV. Exécution des requêtes

Une fois connecté à une base de données via PDO, l'étape suivante consiste à exécuter des requêtes SQL. PDO met à disposition plusieurs méthodes, dont les plus utilisées sont `query()` et `exec()`.

1. Requetes simples : SELECT, INSERT, UPDATE, DELETE

a) Requête SELECT – Lire les données

```
$sql = "SELECT * FROM etudiants";  
$result = $db->query($sql);  
  
foreach ($result as $row) {  
    echo $row['nom'] . " - " . $row['prenom'] . "<br>";  
}
```

- `query()` exécute une requête qui retourne un jeu de résultats (comme un SELECT).
- Le résultat est un objet de type `PDOStatement` qu'on peut parcourir avec `foreach`.

b) Requête INSERT – Ajouter des données

```
$sql = "INSERT INTO etudiants (nom,  
prenom) VALUES ('Ali', 'Benali')";  
  
$nb = $db->exec($sql);  
echo "$nb ligne(s) insérée(s)";
```

- `exec()` exécute une requête qui ne retourne pas de résultats (INSERT, UPDATE, DELETE).
- Elle retourne le nombre de lignes affectées.

IV. Exécution des requêtes

2. Différence entre query() et exec()

Méthode	Utilisation principale	Retourne
query()	Requêtes avec résultats (SELECT)	Un objet PDOStatement (à parcourir)
exec()	Requêtes sans résultats (INSERT, UPDATE, DELETE)	Nombre de lignes affectées

Utilisez toujours query() pour lire des données, et exec() pour modifier des données.

V. Requêtes préparées

1. Pourquoi utiliser des requêtes préparées ?

Les requêtes préparées sont un mécanisme offert par PDO pour sécuriser les interactions avec la base de données, notamment contre les attaques par injection SQL.

❌ Exemple d'injection SQL (à éviter) :

```
$user = $_GET['user'];
```

```
$sql = "SELECT * FROM users WHERE username = '$user'";
```

Si `$user = 'admin' OR '1'='1'`, l'attaque contourne la sécurité et affiche tous les utilisateurs.

V. Requêtes préparées

2. Avantages des requêtes préparées

- ✓ Sécurité renforcée : les données sont séparées du code SQL
- ✓ Performances : une requête préparée peut être exécutée plusieurs fois avec des valeurs différentes
- ✓ Lisibilité et maintenance du code

3. Syntaxe de base : prepare() + execute()

```
$sql = "SELECT * FROM users WHERE username = :username";
```

```
$stmt = $db->prepare($sql); // Prépare la requête
```

```
$stmt->execute([':username' => $user]); // Exécute avec la valeur
```

- :username est un paramètre nommé.
- execute() prend un tableau associatif qui associe les paramètres à leurs valeurs.

V. Requêtes préparées

4. Liaison de paramètres avec bindParam() et bindValue()

bindParam() : Lie une variable au paramètre (la valeur peut changer avant l'exécution).

```
$sql = "INSERT INTO etudiants (nom) VALUES (:nom)";
```

```
$stmt = $db->prepare($sql);
```

```
$nom = "Karim";
```

```
$stmt->bindParam(':nom', $nom);
```

```
$stmt->execute(); // insère "Karim"
```

```
$nom = "Amine";
```

```
$stmt->execute(); // insère "Amine"
```

Bonnes pratiques :

- Toujours utiliser des requêtes préparées pour les données saisies par les utilisateurs.
- Préférer bindParam pour des variables réutilisées, bindValue pour des valeurs fixes.

bindValue() : Lie une valeur immédiate, figée au moment de l'appel.

```
$stmt->bindValue(':nom', 'Sara');
```

```
$stmt->execute(); // insère "Sara"
```

VI. Récupération des résultats

Après avoir exécuté une requête SQL avec PDO, il est souvent nécessaire de récupérer les données renvoyées (par exemple, avec une requête SELECT). PDO fournit plusieurs méthodes pour parcourir les résultats et plusieurs modes de récupération adaptés à différents besoins.

1. `fetch()` : Récupération ligne par ligne

La méthode `fetch()` permet de récupérer une seule ligne à la fois.

```
$sql = "SELECT * FROM users";  
$stmt = $db->query($sql);  
while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {  
    echo $row['username'] . "<br>";  
}
```

Avantage : faible consommation mémoire.

Idéal pour le traitement progressif de gros jeux de données.

VI. Récupération des résultats

2. fetchAll() : Récupération de toutes les lignes d'un coup

```
$sql = "SELECT * FROM users";  
$stmt = $db->query($sql);  
$rows = $stmt->fetchAll(PDO::FETCH_ASSOC);  
foreach ($rows as $row) {  
    echo $row['username'] . "<br>";  
}
```

Avantage : accès direct à tout le jeu de résultats.

À éviter pour des tables très volumineuses (risque de surcharge mémoire).

VI. Récupération des résultats

2. Modes de récupération des résultats

PDO propose plusieurs modes de récupération, selon le format souhaité des lignes :

Mode	Description	Exemple d'accès
PDO::FETCH_ASSOC	Tableau associatif (clé = nom colonne)	<code>\$row['email']</code>
PDO::FETCH_NUM	Tableau indexé numériquement	<code>\$row[0]</code>
PDO::FETCH_BOTH	Tableau associatif + indexé	<code>\$row[0]</code> ou <code>\$row['email']</code>
PDO::FETCH_OBJ	Objet anonyme (stdClass)	<code>\$row->email</code>
PDO::FETCH_CLASS	Objet d'une classe définie	<code>\$user->getName()</code> (avancé)

Exemple avec FETCH_OBJ :

```
$stmt = $db->query("SELECT * FROM users");  
while ($user = $stmt->fetch(PDO::FETCH_OBJ)) {  
    echo $user->username;  
}
```

VI. Cas pratique

Formulaire de connexion utilisateur avec PDO

1. *Script de création de la base ensa et de la table users*

-- Création de la base de données

CREATE DATABASE IF NOT EXISTS ensa ;

-- Utilisation de la base

USE ensa;

-- Création de la table des utilisateurs

*CREATE TABLE users (
 id INT AUTO_INCREMENT PRIMARY KEY,
 username VARCHAR(50) NOT NULL UNIQUE,
 password VARCHAR(255) NOT NULL
);*

-- Insertion d'un utilisateur test (mot de passe : test123)

*INSERT INTO users (username, password)
VALUES ('admin', '\$2y\$10\$i0xT/k/4gfeOcxFguTGj9.aH5ZrFwTLvUDMJrHGIEB58isTtfOwWa ');*

VI. Cas pratique

Formulaire de connexion utilisateur avec PDO

2. Formulaire « form.php »

```
// Connexion à la base
```

```
<!-- Formulaire HTML -->
```

```
<!DOCTYPE html>
```

```
<html lang="fr">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <title>Connexion utilisateur</title>
```

```
</head>
```

```
<body>
```

```
    <h2>Connexion</h2>
```

```
    <form method="POST" action="traitement.php">
```

```
        <label>Nom d'utilisateur : <input type="text" name="username" required></label><br><br>
```

```
        <label>Mot de passe : <input type="password" name="password" required></label><br><br>
```

```
        <button type="submit">Se connecter</button>
```

```
    </form>
```

```
        <?php echo password_hash("test123", PASSWORD_DEFAULT);
```

```
?>
```

```
</body>
```

```
</html>
```

VI. Cas pratique

Formulaire de connexion utilisateur avec PDO

3. Formulaire de traitement « traitement.php »

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {  
    // Connexion à la base de données  
    $dsn = "mysql:host=localhost;dbname=ensa;charset=utf8";  
    $db = new PDO($dsn, 'root', "", [  
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION  
    ]);  
    // Récupération des données du formulaire  
    $username = $_POST['username'] ?? "";  
    $password = $_POST['password'] ?? "";  
  
    // Requête préparée pour éviter les injections SQL  
    $sql = "SELECT * FROM users WHERE username = :username";  
    $stmt = $db->prepare($sql);  
    $stmt->execute(['username' => $username]);  
  
    $user = $stmt->fetch(PDO::FETCH_ASSOC);  
  
    if ($user && password_verify($password, $user['password'])) {  
        echo "<p style='color: green;'>Connexion réussie ! Bienvenue, <strong>" . htmlspecialchars($user['username']) .  
"</strong></p>";  
    } else {  
        echo "<p style='color: red;'>Identifiants incorrects.</p>";  
    }  
}
```



المدرسة الوطنية للعلوم التطبيقية - القنيطرة
National School of Applied Sciences - Kenitra

MERCI